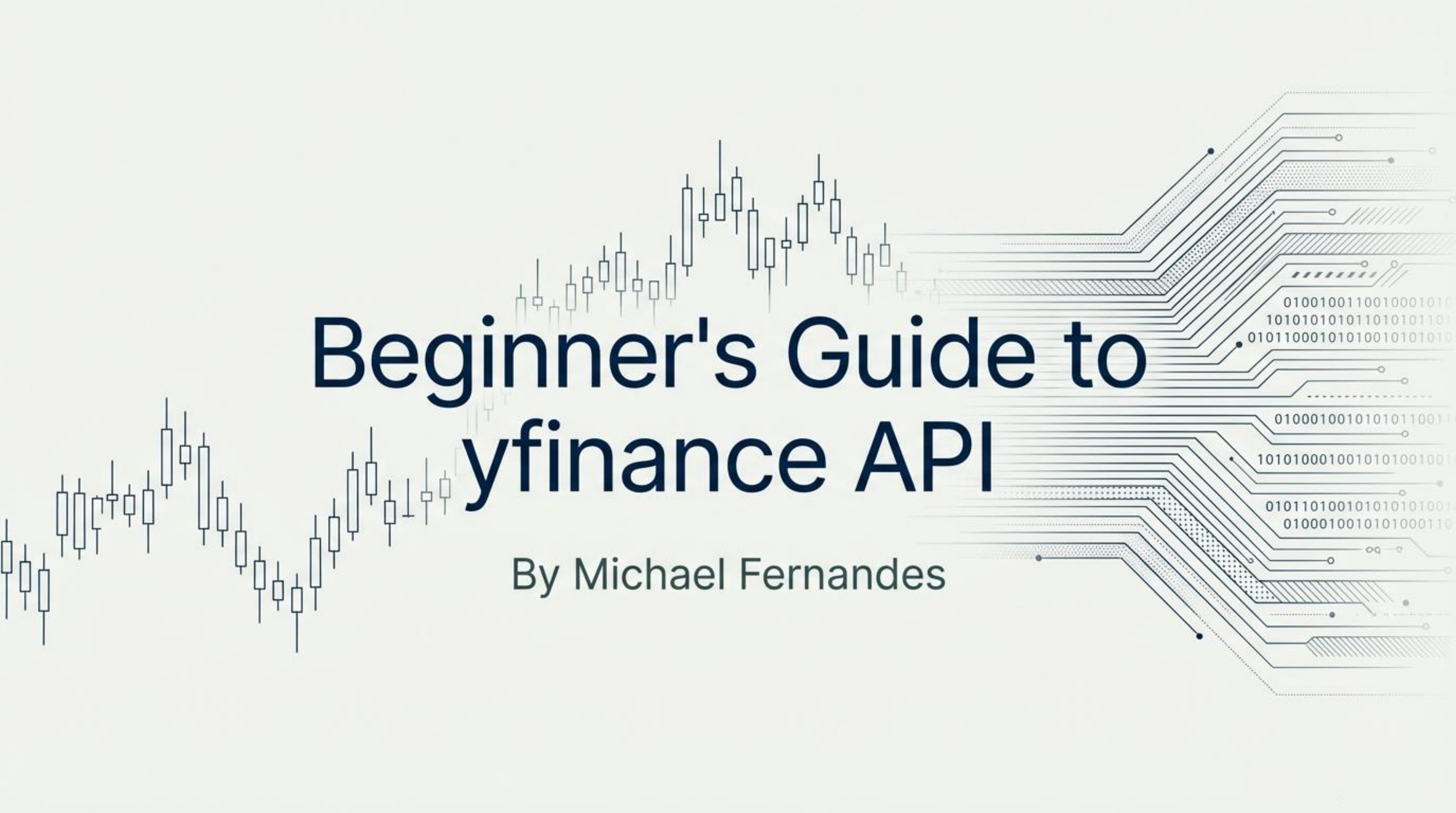




# Beginner's Guide to yfinance API

By Michael Fernandes



# Market Data with Python: The `yfinance` Cheat Sheet

## Core Capabilities

### Simple, Key-Free Setup

```
>_ pip install yfinance
```

Just `pip install yfinance` to start; no API key is needed.

### Fetch Historical Price Data



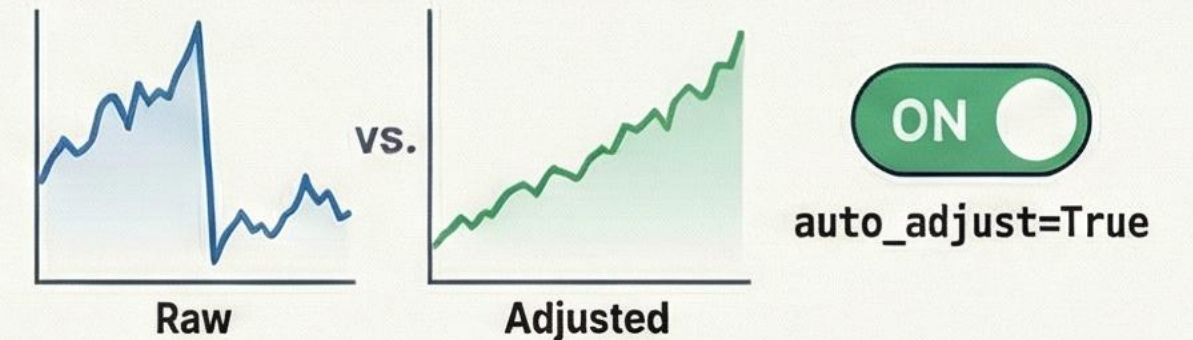
Use `ticker.history()` to get OHLCV data for stocks, crypto, and indices.

### Access Company Fundamentals & More



## Best Practices & Limitations

### Always Use Adjusted Prices for Analysis



Set `auto_adjust=True` to account for dividends and splits in backtesting.

### Be Aware of Common Pitfalls



Missing data



Rate limits



Survivorship bias

Watch for missing data, rate limits, and survivorship bias.

### A Research Sandbox, Not a Trading Engine



Ideal for backtesting and prototyping



use Bloomberg/Refinitiv for production systems.

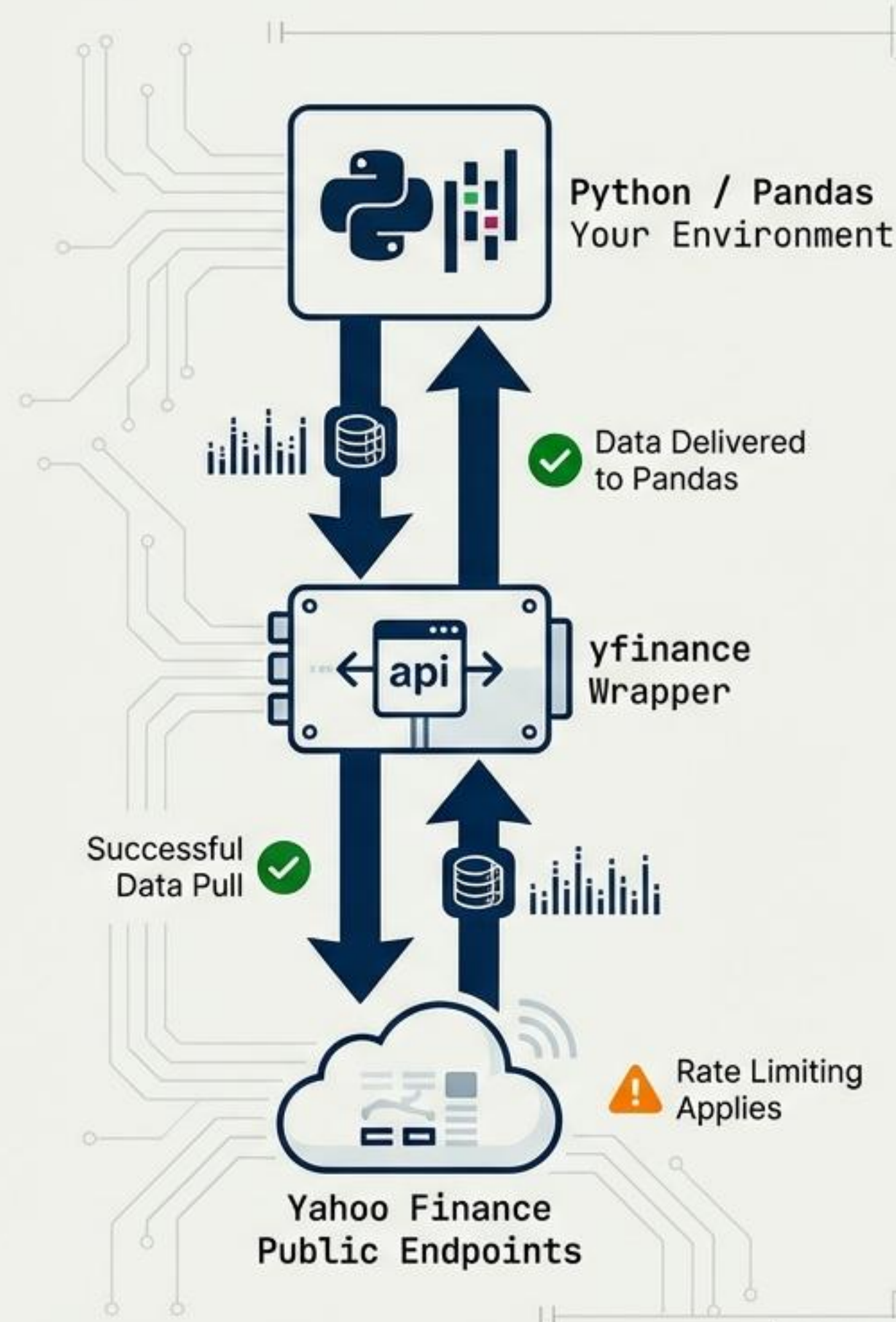


# The Gateway to Market Data

yfinance is the most popular free Python API for pulling market data from public Yahoo Finance endpoints. It acts as a bridge for quants, students, and researchers to perform prototyping and exploratory analysis without expensive subscriptions.

## The Research Sandbox

Ideal for strategy research, factor modeling, and academic projects. Not built for high-frequency trading or production execution.



# The Foundation: Installation & Setup

```
$ pip install yfinance
```



No API key required.  
Immediate access.

```
script.py  
1 import yfinance as yf  
2
```



# The Core Object: The Ticker

Everything in the library revolves around the Ticker object. Instantiating this object connects you to the asset's data streams.

```
script.py x
1 ticker = yf.Ticker('AAPL')
2
```

## Universal Syntax



### Stocks:

AAPL  
RELIANCE.NS



### Indices:

^NSEI  
^GSPC



### Crypto:

BTC-USD



### Forex:

USDINR=X

# Fetching History

The most common use case is retrieving Historical OHLCV prices.

```
# Basic History  
df = ticker.history(period='1y')
```

## Pandas DataFrame Output

| Date       | Open   | High   | Low    | Close  | Volume   |
|------------|--------|--------|--------|--------|----------|
| 2023-10-01 | 171.22 | 173.00 | 170.00 | 172.50 | 51234567 |
| 2023-10-02 | 172.50 | 174.10 | 171.80 | 173.75 | 49876543 |
| 2023-10-03 | 173.75 | 175.20 | 172.90 | 174.90 | 50112233 |
| 2023-10-04 | 174.90 | 176.00 | 173.50 | 175.80 | 48555666 |
| ...        | ...    | ...    | ...    | ...    | ...      |



# Controlling Time: Intervals & Ranges

```
script.py

df = ticker.history(
    start='2022-01-01',
    end='2024-01-01',
    interval='1d'
)
```

## Supported Intervals

| Interval | Use Case                           |
|----------|------------------------------------|
| 1m, 5m   | Intraday (Limit: ~60 days history) |
| 1h       | Swing trading                      |
| 1d       | Standard strategies                |
| 1wk      | Long-term analysis                 |
| 1mo      | Macro / Asset allocation           |

# Decoding the Output: OHLCV

**df.columns** returns the standard financial data points.





# Crucial for Quants: Adjusted Prices



Warning: Unadjusted data invalidates backtesting.

```
script.py  
  
df = yf.download('AAPL', auto_adjust=True)
```

This parameter automatically adjusts historical data for:

- Dividends
- Stock Splits





# Scaling Up: Bulk Downloads

Efficiently retrieve data for portfolios or indices by passing a list of tickers.

```
script.py

tickers = ['AAPL', 'MSFT', 'GOOGL']
data = yf.download(tickers, start='2023-01-01')
```

`data['Close']['AAPL']`

data['Close']

| AAPL   | MSFT  | GOOGL |
|--------|-------|-------|
| 175.50 | 93.00 | 99.02 |
| 175.50 | 93.50 | 99.07 |
| 175.50 | 98.00 | 90.90 |
| AAPL   | MSFT  | GOOGL |
| 175.50 | 99.30 | 99.73 |
| 175.50 | 95.80 | 99.03 |
| 175.50 | 98.50 | 90.20 |
| 175.50 | 98.00 | 90.07 |
| 175.50 | 99.30 | 99.50 |
| 175.50 | 95.80 | 90.80 |





# Beyond Price: Company Fundamentals



script.py

```
# Annual Data
ticker.income_stmt
ticker.balance_sheet
ticker.cashflow

# Quarterly Data
ticker.quarterly_income_stmt
```



| BALANCE SHEET                          |                    |
|----------------------------------------|--------------------|
| <b>Assets</b>                          |                    |
| Total Current Assets:                  | \$135,400 M        |
| Cash & Equivalents:                    | \$28,300 M         |
| Total Non-Current Assets:              | \$245,100 M        |
| <b>Total Assets:</b>                   | <b>\$380,500 M</b> |
| <b>Liabilities &amp; Equity</b>        |                    |
| Total Current Liabilities:             | \$120,200 M        |
| Total Non-Current Liabilities:         | \$150,500 M        |
| <b>Total Liabilities:</b>              | <b>\$270,700 M</b> |
| Total Stockholders' Equity:            | \$109,800 M        |
| <b>Total Liabilities &amp; Equity:</b> | <b>\$380,500 M</b> |



# Valuation & Metadata

The 'info' dictionary provides a snapshot of current valuation metrics and sector details.

```
info = ticker.info
```

**marketCap**

**trailingPE**

**beta**

**sector**

**industry**

**dividendYield**

*Pro Tip: 'info' calls can be slow. Always cache these results locally.*



# Corporate Actions: Dividends & Splits

Track cash payouts and structural changes. Essential for Total Return calculations.



```
ticker.dividends  
ticker.splits
```



*Without accounting for these actions,  
price charts show false losses.*



# The Hidden Gem: Options Data

yfinance retrieves full option chains, including Volatility and Open Interest.

```
# Get expirations
ticker.options

# Load chain
opt = ticker.option_chain('2024-12-20')
calls = opt.calls
puts = opt.puts
```

| Contract           | Strike | Bid   | Ask   | Vol | Open Int | Implied Vol |
|--------------------|--------|-------|-------|-----|----------|-------------|
| XYZ241220C00150000 | 150.00 | 12.50 | 13.10 | 450 | 1200     | 34.5%       |
| XYZ241220C00155000 | 155.00 | 9.20  | 9.80  | 300 | 850      | 35.2%       |
| XYZ241220C00160000 | 160.00 | 6.50  | 7.00  | 220 | 600      | 36.1%       |
| XYZ241220P00150000 | 150.00 | 5.30  | 5.80  | 180 | 450      | 34.5%       |
| XYZ241220P00155000 | 155.00 | 7.90  | 8.40  | 210 | 520      | 35.2%       |
| XYZ241220P00160000 | 160.00 | 11.10 | 11.70 | 260 | 680      | 36.1%       |



# Asset Universality

Use the correct suffix/symbol format for global asset classes.

## Indices

NIFTY 50 (^NSEI), S&P 500 (^GSPC)

## Commodities

Gold (GC=F)

## Crypto

Bitcoin (BTC-USD)

## Forex

USD/INR (USDINR=X)

```
yf.download('BTC-USD', period='5y')
```



# Technical Indicators (The DIY Approach)

yfinance provides raw data.  
Use 'pandas' or 'ta' libraries to  
calculate indicators.



```
import ta

# RSI Calculation
df['rsi'] = ta.momentum.RSIIndicator(df['Close']).rsi()

# Simple Moving Average
df['sma_50'] = df['Close'].rolling(50).mean()
```



# Critical Pitfalls (Safety Inspection)



## **Holidays**

Data may be missing/null on market holidays.



**Survivorship Bias** - Delisted companies are often unavailable.



**Rate Limits** - Aggressive loops can result in IP blocks.



**Intraday Instability** - Intraday history is volatile and limited.



# yfinance vs. The Giants

| Feature      | yfinance         | Bloomberg/Refinitiv  |
|--------------|------------------|----------------------|
| Cost         | Free             | \$\$\$\$             |
| Real-time    | Delayed          | Yes                  |
| Fundamentals | Inconsistent     | Institutional Grade  |
| Use Case     | Research Sandbox | Production & Trading |



# Best Practices for Stability



**Cache Locally** - Save to CSV/SQL to avoid re-downloading.



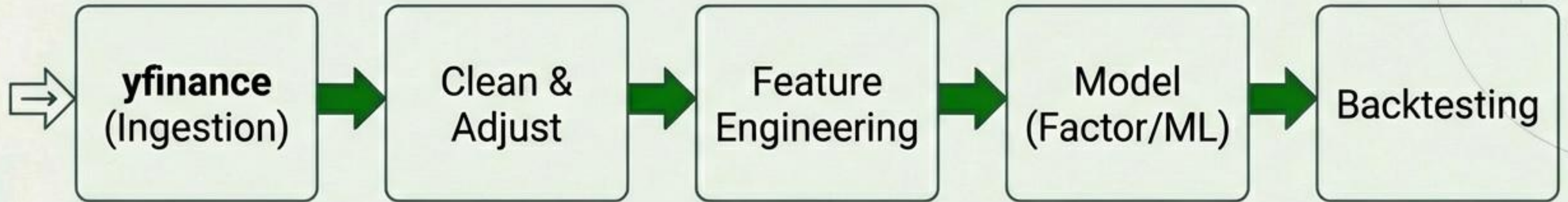
**Validate** - Cross-check data points against secondary sources.



**Research Only** - Do not hook directly into automated execution systems.



# The Quant Workflow



**yfinance** is the starting point for modern Factor Investing and ML pipelines.